

综合实战项目

训练和部署程序简介

1. 训练过程

阶段零 · 环境与模型准备

- **硬件假设**: 单卡 8GB+ 显存 (对应脚本里 BATCH_SIZE=4、MAX_LENGTH=512 的默认值)
 - **依赖安装**: transformers、peft、datasets、accelerate、torch、bert_score、jieba; 量化阶段额外需要 transformers==4.51.3 + autoawq==0.2.9
 - **下载基座模型**: 在项目根目录执行 sh ./download.sh, 把 Qwen1.5-1.8B-Chat 和 bge-m3 下载到 models/ 目录
 - **准备 API Key**: export DEEPSEEK_API_KEY="sk-..." (或 OPENAI_API_KEY), 用于后续造数据
-

阶段一 · 训练数据生成 (第 1 步)

脚本: 1.generate_customerservice_data.py

- **目标**: 通过教师模型 (DeepSeek/OpenAI) 批量生成高质量客服问答对
 - **产物**: customerservice_alpaca.json (Alpaca 格式)
 - **数据类别设计** (三类样本平衡):
 - **normal**: 常规客服问答 (商品咨询、售后政策等)
 - **rag**: 带 **【系统检索结果】** 前缀、携带结构化订单/物流/退款数据的样本——训练模型**忠实引用检索到的数字和单号**, 不自己编
 - **red**: 红队/越狱样本, 训练模型在遇到角色替换、规则绕过时**正确拒绝**
 - **运行**: python 1.generate_customerservice_data.py --provider deepseek --total 2000
-

阶段二 · 格式转换 (第 2 步)

脚本: 2.convert_to_chatml.py

- **目标：**把 Alpaca 的 {instruction, input, output} 转成 Qwen 官方的 ChatML messages 格式
- **产物：**train.json、test.json
- **格式示例：**

json

```
{
  "messages": [
    {
      "role": "system",
      "content": "你是专业电商客服助手..."
    },
    {
      "role": "user",
      "content": "【系统检索结果】...+ 用户问题"
    },
    {
      "role": "assistant",
      "content": "客服回复"
    }
  ]
}
```

- **运行：**python 2.convert_to_chatml.py --input customerservice_alpaca.json --output train.json

阶段三 · LoRA 微调（第 3 步，核心训练步）

脚本：3.finetune_qwen_customerservice.py

- **目标：**在基座 Qwen1.5-1.8B-Chat 上做**参数高效微调**，只训练 LoRA adapter（约几十 MB），不动 18 亿基座权重
- **关键超参**（环境变量可覆盖）：

参数	默认值	作用
MAX_LENGTH	512	最大序列长度，RAG 场景可上调
LORA_RANK	16	LoRA 秩，16 是质量与显存的平衡点
BATCH_SIZE	4	单卡微批次
GRAD_ACCUM	4	梯度累积，等效批次 = 4×4 = 16
EPOCHS	3	训练轮数

- **训练要点：**只对 assistant 部分的 token 计算 loss，user/system 部分被 mask（标准 SFT 做法）
- **产物：**lora-adapter/（只含 LoRA 权重 + 配置）

- 运行：python 3.finetune_qwen_customerservice.py

阶段四 · 微调效果评估（第 4 步，质量闸口）

脚本：4.evaluate_models.py（准确度）或 4.compare_models.py（内存和速度）

- 目标：量化验证微调是否真的有效，拿到走下一步的通行证
- 对比：基座模型 vs 微调模型（LoRA 加载后）
- 四个评估维度：

指标	对应样本类别	说明
BERTScore F1	全部	与参考答案的语义相似度，用本地 Qwen 计算
字数合规率	normal	回答 ≤ 150 字的比例
拒绝准确率	red	越狱样本是否正确拒绝
RAG 数字忠实度 rag		回答中的订单号、金额是否都来自检索结果

- 样本类别自动判断：含 【系统检索结果】 → rag；含越狱关键词 → red；其他 → normal
- 实用参数：
 - --max-cases 20：小样本快速验证
 - --inference-cache cache.json：跳过推理、只重跑评估
- 决策点：如果微调模型在四项指标上没显著优于基座，回阶段一检查数据质量或阶段三调超参

阶段五 · LoRA 权重合并（第 5 步）

脚本：5.merge_lora_model.py

- 目标：把 LoRA adapter 数学合并进基座权重，得到独立可部署的完整模型（不再需要 PEFT 运行时）
- 为什么要合并：vLLM 虽然支持 LoRA 热加载，但不支持 LoRA + 量化组合——要走量化部署路线，必须先合并
- 产物：merged-model/（FP16 完整权重，体积 ≈ 3.5GB）

- 运行:

bash

```
python 5.merge_lora_model.py \
```

```
--base ../models/Qwen1.5-1.8B-Chat \
```

```
--adapter ./lora-adapter \
```

```
--output ./merged-model
```

阶段六 · AWQ 4bit 量化 (第 6 步)

脚本: 6.awq_quantize.py

- 目标: 把 FP16 模型量化到 4bit, **模型体积缩小 4 倍、推理速度提升、显存占用大幅下降**
- 为什么选 **AWQ**: 激活感知量化, 对客服这类稳定分布场景精度损失很小; vLLM 原生支持
- 关键参数:
 - --n-samples 128: 校准样本数, 用于统计激活分布
- 产物: awq-model/ (4bit 量化权重, 约 1GB)
- 运行: `python 6.awq_quantize.py --input ./merged-model --output ./awq-model --n-samples 128`

阶段七 · 量化模型双维度评估 (第 7 步, 上线闸口)

脚本: 7.evaluate_quantization.py (准确度) 7.compare_quantization.py (内存和速度)

- 目标: 确认量化没破坏模型能力、同时量化带来的性能收益真实存在
- 对比: merged-model (FP16) vs awq-model (AWQ 4bit)
- 双维度评估:

维
度 指标

质 BERTScore、字数合规率、拒绝准确率、RAG 忠实度 (沿用第 4 步的评估体
量 系)

维度	指标
性能	首 Token 延迟 (TTFT)、生成速度 (tokens/s)、显存占用、模型体积
	• 输出: 完整对比报告, 包含质量结论 + 性能结论 • 决策点: 质量下降 $\leq 2\%$ 且性能显著提升, 则放行上线; 否则考虑升到 8bit 或换量化方法

阶段八 · 交付部署

- **拷贝产物**: 在项目根目录执行 `cp -r train/awq-model/* models/merged-model/` (或直接改 `deploy/config.py` 里的 `MODEL_PATH`)
- **启动服务**: `cd deploy && sh ./start_all.sh`
- **冒烟测试**: 调 `/health` 看 vLLM 和 Redis 状态, 再打一条带 RAG 触发词的请求到 `/v1/chat/stream`

关键原则

整条流水线体现的工程思路是**每一步都有产物、每两步之间都有评估闸口**: 第 4 步卡住“微调是否有效”, 第 7 步卡住“量化是否安全”。一旦某个指标退化, 可以明确回溯到具体阶段, 而不是推倒重来。

2. 部署过程

阶段零 · 前置条件

- **训练产物就绪**: `train/awq-model/` (AWQ 4bit 量化模型)
 - **基础设施**: Redis (默认 `localhost:6379`)、Python 3.10+、单卡 GPU (1.8B 模型量化后约 1-2GB 显存)
 - **向量模型**: `models/bge-m3/` (RAG 检索用)
 - **PDF 知识库**: 产品手册 / 平台政策 PDF, 用于 RAG 兜底 (项目自带 `产品手册_电商平台.pdf`)
-

阶段一 · 模型产物就位

- 拷贝量化模型到部署目录：

```
bash

# 在项目根目录执行

cp -r train/awq-model/* models/merged-model/
```

- 或修改配置指向训练目录：在 deploy/config.py 设置 MODEL_PATH 环境变量
- 验证文件完整性：确认 config.json、tokenizer.json、权重文件 (.safetensors) 齐全

阶段二 · 依赖安装

```
bash

cd deploy

pip install -r requirements.txt
```

核心依赖及其作用：

依赖	作用
vllm>=0.4.0	推理引擎，原生支持 AWQ 量化
fastapi	Web 网关框架
chromadb	向量数据库（RAG 存储）
sentence-transformers	加载 BGE-M3 做向量化
redis[asyncio]	异步会话存储
gradio	演示界面
prometheus-fastapi-instrumentator	监控指标

阶段三 · 配置管理 (config.py)

所有配置通过**环境变量**覆盖默认值，这是生产部署的核心：

3.1 必改配置（生产环境）

环境变量	说明	示例
PRODUCTION	启用生产模式检查	true
API_KEYS	API 密钥，多个逗号分隔	key-abc,key-def
CORS_ORIGINS	允许的跨域域名	https://your-domain.com
MODEL_PATH	量化模型路径	./models/merged-model
EMBED_MODEL	向量模型路径	./models/bge-m3

注意： PRODUCTION=true 时如果没设 API_KEYS，应用会启动失败——这是有意的强约束

3.2 性能调优配置

环境变量	默认	作用
RATE_LIMIT	20	每分钟请求上限（每个 Key 独立计数）
RATE_BURST	5	突发额外配额
HTTP_TIMEOUT	60	vLLM 请求超时（秒）
RAG_TOP_K	3	RAG 检索召回数
RAG_DISTANCE_THRESHOLD	0.6	余弦距离阈值，超过则视为无结果

3.3 端口分配

服务	默认端口	环境变量
vLLM 推理	8001	VLLM_PORT
FastAPI 网关	8000	API_PORT
Gradio 前端	7860	GRADIO_PORT

阶段四 · RAG 知识库初始化

如果用到 **PDF 兜底检索**，首次启动前需要把 PDF 灌入 ChromaDB：

- **入库逻辑：** rag.py 的 ingest_pdf() 函数，用 pdfplumber 抽文本、按 400 字符分块、50 字符重叠

- **向量化**: BGE-M3 生成 embedding, 存入 ChromaDB (余弦相似度)
 - **路径白名单**: PDF_UPLOAD_DIRS=/tmp,./uploads 防止路径遍历攻击
 - **验证入库**: 启动后调 /health, 返回的 kb_docs 字段是入库文档数, 应大于 0
-

阶段五 · 启动服务 (start_all.sh)

bash

cd deploy && sh ./start_all.sh

脚本分三步启动, 每一步都有健康检查:

5.1 启动 vLLM 推理服务 (端口 8001)

- 以 AWQ 量化模式加载 MODEL_PATH 下的模型
- 启动后 vLLM 暴露 OpenAI 兼容的 /v1/chat/completions 接口
- **典型启动命令** (脚本内部):

bash

python -m vllm.entrypoints.openai.api_server \

--model \$MODEL_PATH \

--quantization awq \

--port 8001

- **健康检查**: 轮询 http://localhost:8001/health 直到返回 200

5.2 启动 FastAPI 网关 (端口 8000)

- uvicorn app.main:app --workers 2, 两个 worker 提升并发
- 启动时初始化: Redis 连接池、ChromaDB、BGE-M3 模型 (带线程锁防止多 worker 重复加载)
- **健康检查**: curl http://localhost:8000/health, 期望返回:

json

```
{"status": "ok", "vllm": true, "redis": true, "kb_docs": N}
```

5.3 启动 Gradio 前端 (端口 7860, 可选)

- 仅用于演示和调试, 生产环境可不启动

- 直接调用 FastAPI 网关，不绕过鉴权和限流
-

阶段六 · 冒烟测试

6.1 健康检查

bash

```
curl http://localhost:8000/health
```

任意一项为 false 都要先排查，不要直接上流量。

6.2 流式对话测试

bash

```
curl -X POST http://localhost:8000/v1/chat/stream \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your-key" \
  -d '{"user_id": "test", "message": "我的订单 ORD-2024-00001 什么时候发货?"}'
```

应触发的完整链路：

1. API Key 鉴权通过
2. 限流计数 +1
3. query_order.py 识别出"订单"意图 + 订单号
4. 查 mock_db 返回订单状态
5. 注入 【系统检索结果】 到 system prompt
6. vLLM 流式生成回复
7. 后台异步保存到 Redis

6.3 RAG 兜底测试

bash

```
curl ... -d '{"message": "你们的退货政策是怎样的?"}'
```

这条不会命中订单数据库，会走 ChromaDB 检索 PDF。日志里应看到 rag=yes。

6.4 关键日志观察点

main.py 会输出两条关键日志：

- TTFT=XXms rag=yes/no: 首 token 延迟, RAG 是否命中
 - total=XXms tokens≈N: 端到端延迟和生成长度
-

阶段七 · 生产加固

7.1 安全

- **API Key 轮换**: 通过 API_KEYS 环境变量支持多 Key 并存, 逐步淘汰旧 Key
- **CORS 收紧**: 生产环境禁止使用 *, 必须写具体域名
- **PDF 上传白名单**: 限制在 PDF_UPLOAD_DIRS 内, 强制 .pdf 扩展名, 已有路径遍历防护
- **全局异常处理器**: 未捕获异常统一返回 500, 不泄漏内部堆栈

7.2 可观测性

- **Prometheus 指标**: /metrics 端点已由 Instrumentator 自动暴露
- **日志**: 同时写 stdout 和 api.log 文件, 方便 tail -f 和日志收集
- **关键监控指标**: 请求 QPS、p95 延迟、TTFT、vLLM/Redis 健康状态、限流触发次数

7.3 并发与容错

代码里已经处理的三个并发点:

- **Redis 连接**: asyncio.Lock 保护懒初始化, 避免多 worker 重复建连
- **向量模型**: threading.Lock 保护单例加载
- **后台保存任务**: asyncio.create_task + 异常回调, 不阻塞响应流且异常不会静默丢失

降级策略:

- **Redis 挂了**: 限流放行、会话不保存, 返回正常回复
 - **vLLM 挂了**: 返回 [ERROR] 推理服务暂时不可用, 不让连接挂死
 - **RAG 向量模型加载失败**: 降级到纯 LLM, 不走检索
-

阶段八 · 停止与重启

bash

优雅停止（按反向顺序：Gradio → FastAPI → vLLM）

sh ./stop_all.sh

更新模型后重启

sh ./stop_all.sh && sh ./start_all.sh

热更新的注意点：vLLM 模型加载需要几十秒到几分钟，重启期间服务不可用。生产环境需要蓝绿部署或双实例滚动。

阶段九 · 运维接口速查

接口	方法	用途
/v1/chat/stream	POST	流式对话主接口
/v1/session/{id}	DELETE	清除指定会话的历史
/health	GET	vLLM + Redis + 知识库状态
/metrics	GET	Prometheus 监控指标

关键原则

整套部署设计的核心思路是**每一层都可独立降级**：vLLM 不可用时网关返回友好错误不挂起；Redis 不可用时限流和会话降级但对话继续；RAG 失败时走纯 LLM。配合 /health 的多维度检查和 Prometheus 指标，出问题能定位到具体组件，而不是整个服务黑盒崩溃。